Artificial Neural Networks and Deep Architectures, DD2437

Lab assignment 1b: Learning with backpropagation and generalisation in multi-layer perceptrons

Livia Cardaccia, Giovanni Prete

September 23, 2026

1 Main objectives and scope of the assignment

The primary objectives of this assignment are to:

- Implement and train a Multi-Layer Perceptron (MLP) using backpropagation to classify linearly non-separable data and analyze the impact of sampling bias on generalization.
- Investigate representation learning and auto-association through the encoder problem.
- Approximate a continuous non-linear Gaussian function to evaluate the effects of network capacity, training set sparsity, and learning parameters on convergence.
- Perform chaotic time-series forecasting (Mackey-Glass equation) using time-delay embeddings.
- Evaluate the effects of architectural capacity, early stopping, and L_2 penalty regularization on robustness to artificial noise and random weight initialization.

The scope encompasses evaluating both clean and artificially noisy datasets to simulate real-world conditions, utilizing strictly consecutive data splitting in time-series tasks to prevent temporal data leakage during evaluation.

2 Methods

For Part I (Classification and Function Approximation), the MLP and backpropagation algorithms were implemented entirely from scratch using standard

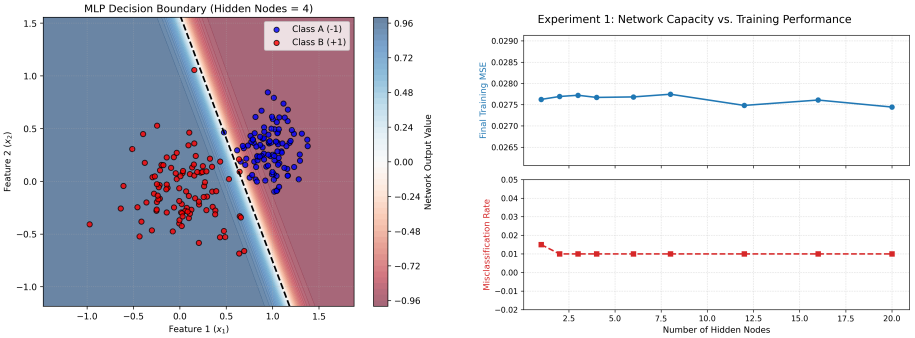
Python and NumPy to handle matrix operations. For Part II (Time-Series Prediction), the Keras deep learning framework, with PyTorch as the backend, was utilized to construct the architectures and manage optimization dynamics. Throughout both sections, the primary objective function and performance metric evaluated was the Mean Squared Error (MSE).

3 Results and discussion

3.1 Classification and regression with a two-layer perceptron

3.1.1 Classification of linearly non-separable data

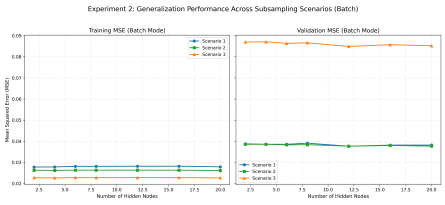
As a baseline, the network was trained on the full, unbiased dataset to determine the minimum parametric capacity required for perfect linear separation (Figure 1).



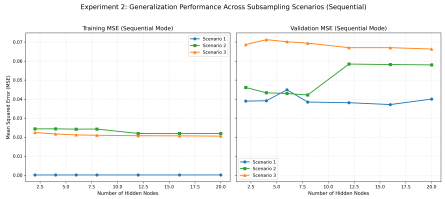
(a) Decision boundary (full dataset). (b) MSE vs. hidden nodes.

Figur 1: Baseline classification performance on the full linearly non-separable dataset.

The network's generalization capability was then tested across three subsampling scenarios, ranging from uniform class representation (Scenario 1) to severe representation bias (Scenario 3). A comparison between batch and sequential training modes revealed distinct optimization dynamics (Figure 2 and Figure 3).



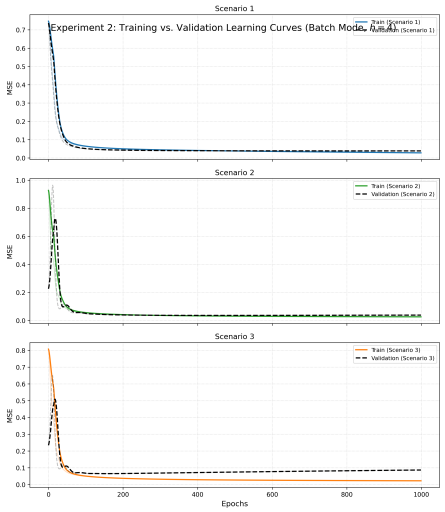
(a) Batch mode capacity comparison.



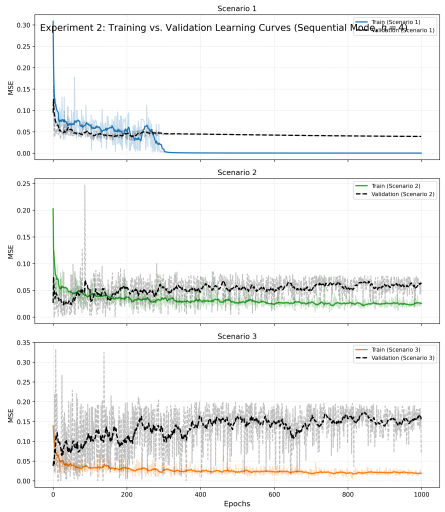
(b) Sequential mode capacity comparison.

Figur 2: Generalization performance across subsampling scenarios.

Batch mode demonstrated smooth, deterministic convergence, settling into stable local minima across all scenarios. On the contrary, sequential mode exhibited high-variance, stochastic updates within each epoch.



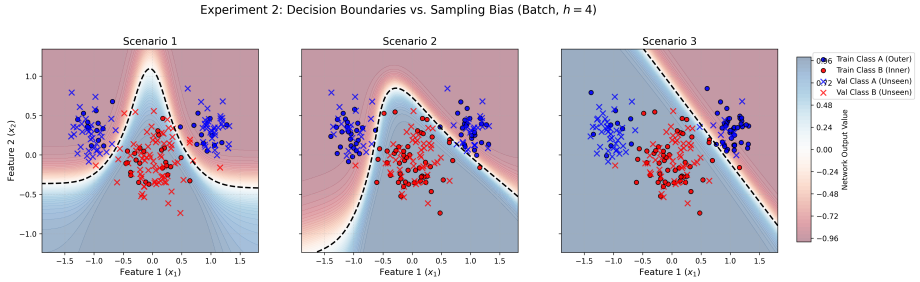
(a) Batch mode learning curves.



(b) Sequential mode learning curves.

Figur 3: Training and validation optimization dynamics under biased sampling.

This stochastic behavior heavily impacted generalization under biased sampling. In Scenario 3, where the training set retained only 20% of a specific class subclass, the sequential mode aggressively overfitted the biased data. This resulted in a diverging validation curve, where validation error increased as training progressed. Decision boundary visualizations (Figure 4) confirmed this behavior: the network physically distorted its separation boundary to fit the skewed training distribution, resulting in widespread misclassification of the unseen validation samples in the underrepresented region.



Figur 4: Decision boundaries adapting to biased subsampling scenarios (batch mode, $h = 4$). Unseen validation points are marked with 'x'.

3.1.2 Function approximation

We investigated the ability of the MLP to approximate the Gaussian function $f(x, y) = e^{-(x^2+y^2)/10} - 0.5$. The function was sampled on a 20×20 grid, resulting in 400 input-output pairs. A baseline 2–20–1 network with $\eta = 0.2$ and $\alpha = 0.9$ achieved an all-data MSE of 1.09×10^{-4}.

The effect of the hidden-layer size was evaluated using between 1 and 25 hidden nodes. Very small networks performed poorly, with MSEs of 0.0625 and 0.0379 for one and two hidden nodes, respectively. Performance improved substantially with more hidden nodes, reaching 6.17×10^{-5} with 25 hidden nodes. The relationship was not strictly monotonic, however, which is consistent with the stochastic nature of the training process. The 2–25–1 architecture was therefore selected for the following experiments.

Hidden nodes	MSE	Epochs (MSE < 0.01)
1	0.062529	–
2	0.037919	–
3	0.002137	331
5	0.000308	192
10	0.000187	108
15	0.000284	91
20	0.000109	77
25	0.000062	50

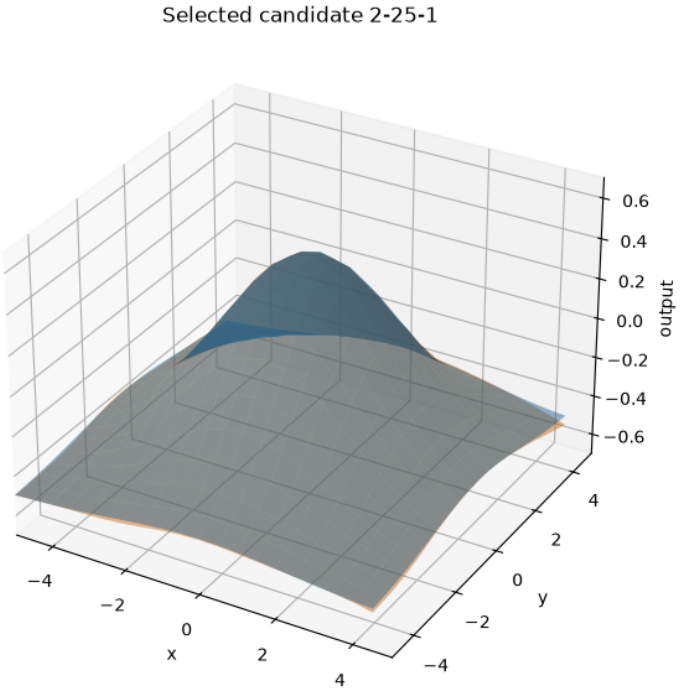
Tabell 1: Effect of hidden-layer size on function approximation.

For the selected architecture, the training-set size was reduced from 80% to 20% of the available samples. The all-data MSE remained almost unchanged between 80% and 60% training data, but increased as fewer samples were available, reaching 2.28×10^{-4} with only 20% of the samples. This indicates that the network generalises well with a sufficiently large training subset, while performance deteriorates when the available training data becomes too sparse.

Training samples	Training MSE	All-data MSE
80% (320)	0.0000569	0.0000585
60% (240)	0.0000609	0.0000622
40% (160)	0.0000728	0.0000857
20% (80)	0.0000839	0.0002279

Tabell 2: Effect of training-set size on approximation and generalisation.

Finally, the convergence speed was investigated by varying the learning rate and momentum coefficient. Increasing the momentum consistently accelerated convergence. The baseline configuration reached MSE < 0.01 after 77 epochs, whereas $\eta = 0.3$ and $\alpha = 0.9$ reached the same threshold after only 36 epochs, while also obtaining a lower final MSE (4.84×10^{-5}). Thus, in the tested configurations, convergence could be accelerated without degrading the final approximation performance.



Figur 5: Target Gaussian function and its approximation obtained with the selected MLP.

3.2 Multi-layer perceptron for time series prediction

3.2.1 Three-layer perceptron for time series prediction – model selection, validation

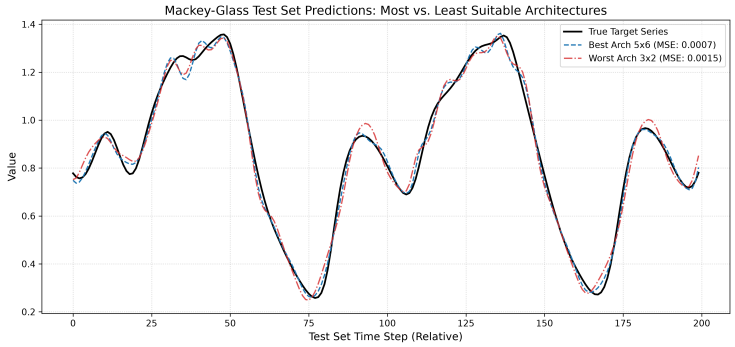
The Mackey-Glass chaotic time series was generated and embedded into input-output pairs utilizing five time-delayed observations $(t-20,\ldots,t)$ to predict $x(t+5)$. To definitively prevent temporal data leakage (look-ahead bias) during evaluation, the 1200 embedded samples were split into strictly consecutive blocks: 700 for training, 300 for validation, and the final 200 for testing.

A grid search was conducted over a Cartesian product of hidden layer dimensions ($nh_1 \in \{3,4,5\}, nh_2 \in \{2,4,6\}$). To assess the robustness of each architecture to stochastic weight initialization, each configuration was evaluated across 10 random seeds. Early stopping monitored on the validation set was utilized to prevent premature convergence and overfitting.

As detailed in Table 3, when allowed sufficient epochs to converge, higher-capacity models naturally dominated. The 5×6 architecture was identified as the most suitable, yielding the lowest mean validation MSE (0.0007) with high stability. The 3×2 architecture proved the least suitable (0.0015), lacking the parametric capacity to consistently map the complex phase-space dynamics. Figure 6 visualizes the performance of the 5×6 and 3×2 architectures on the strictly held-out test set, confirming the generalization superiority of the higher-capacity configuration.

n_{h1}	n_{h2}	Val. MSE Mean	Val. MSE S.D.	Test MSE Mean	Test MSE S.D.
5	6	0.000705	0.000244	0.000779	0.000218
4	4	0.000915	0.000261	0.000812	0.000304
3	4	0.000935	0.000180	0.000827	0.000212
5	4	0.000972	0.000253	0.000855	0.000278
4	6	0.001095	0.000385	0.000989	0.000386
4	2	0.001125	0.000332	0.000911	0.000232
3	6	0.001151	0.000382	0.001045	0.000405
5	2	0.001278	0.000474	0.001169	0.000555
3	2	0.001580	0.000417	0.001358	0.000364

Tabell 3: Grid search over hidden-layer sizes averaged across 10 random weight initializations.



Figur 6: Network predictions vs. true Mackey-Glass values on the unseen test set for the most (5×6) and least (3×2) suitable architectures.

3.2.2 Three-layer perceptron for noisy time series prediction with penalty regularisation

To simulate realistic conditions, zero-mean Gaussian noise ($\sigma \in \{0.05, 0.15\}$) was injected exclusively into the training feature set. Early stopping was disabled to isolate the effect of L_2 weight decay regularization (λ) on a fixed $n_{h1} = 5$ architecture.

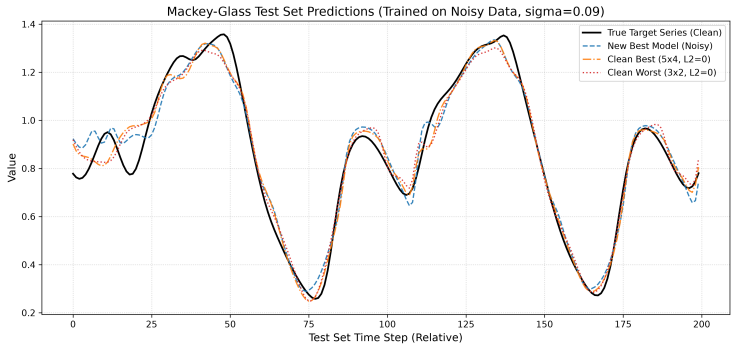
As observed in Table 4, the introduction of even a mild penalty ($\lambda = 0.001$) successfully collapsed the mean L_2 norm of the network weights from ≈ 35.8 down to ≈ 12.8 (for $\sigma = 0.15, n_{h2} = 6$). However, this aggressive weight shrinkage did not translate to improved validation performance. Instead, increasing λ beyond 0.001 caused severe underfitting, driving the validation error up by an order of magnitude. This indicates that, in the case of the highly nonlinear Mackey-Glass system, strict parameter regularization deprives the network of the ability to model the underlying dynamics, making standard early stopping a far superior regularizer for this specific task.

σ	n_{h2}	λ	Val. MSE Mean	Mean Weight Norm
0.05	6	0.000	0.001826	34.00
0.05	6	0.001	0.003651	14.14
0.05	6	0.010	0.011557	15.05
0.05	6	0.100	0.016707	23.18
0.15	6	0.000	0.007141	35.79
0.15	6	0.001	0.009328	12.88
0.15	6	0.010	0.015006	14.15
0.15	6	0.100	0.021705	21.37

Tabell 4: Effect of regularisation strength on validation error and weight norm (fixed $n_{h1} = 5$).

For the intermediate noise scenario ($\sigma = 0.09$), the clean validation subset was utilized to select the optimal model configuration from scratch. The validation set was strictly preferred for model selection as evaluating on the noisy training set inherently favors overfitted models, while utilizing the test set introduces data leakage.

When retrained from scratch on the noisy data using early stopping ($\lambda = 0$), the newly selected Best Noisy Model achieved a Test MSE of 0.00303 ± 0.0002. The original Clean Best (5×6) achieved a nearly identical 0.00309 ± 0.0002, while the Clean Worst (3×2) degraded to 0.00358 ± 0.0003. Notably, training on corrupted data caused a $\approx 4\times$ degradation in generalization performance compared to the clean baseline results (0.00078).



Figur 7: Test set predictions comparing the optimal model selected on noisy data ($\sigma = 0.09$) against the clean-data architectures.

4 Final remarks

During the experiments, the tension between architectural capacity and regularization became clear. In the classification task, subsampling revealed how quickly stochastic training paradigms adapt to distorted data when not subject to constraints. In time series forecasting, the mathematical isolation of the validation set prevented early temporal distortion, ensuring that our architectural choices generalized to truly unseen sequences. It is interesting to note that, while L_2 regularization effectively constrained the weight distributions in the presence of noise, it led to severe underfitting in the chaotic time series task, demonstrating that dynamic early stopping is often a more appropriate regularizer for preserving the capacity needed to map highly nonlinear phase spaces.